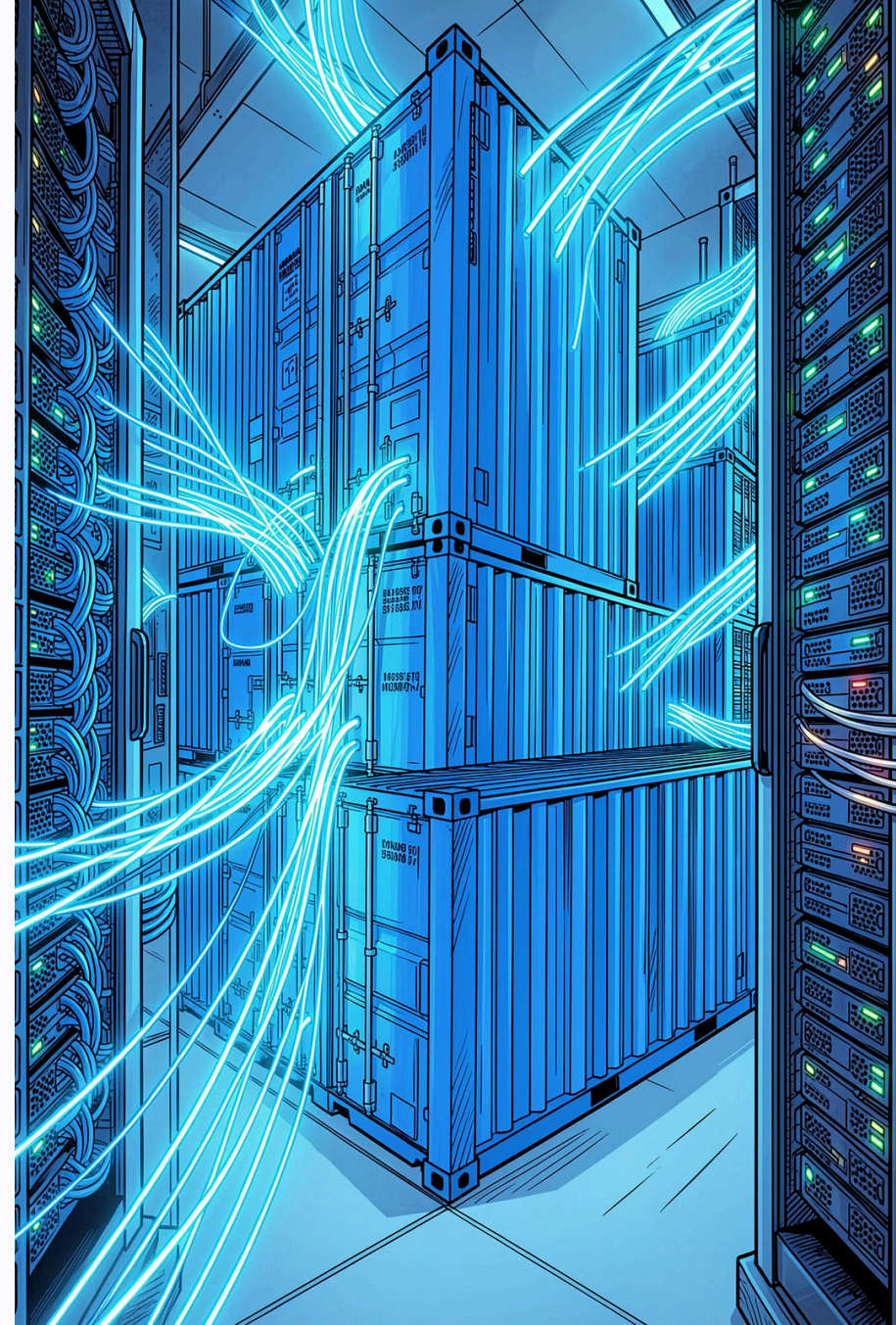


OPENSIFT CONTAINER PLATFORM · MODULE 1

Container Fundamentals & Runtime Refresher

Containers standardize application packaging and execution — making applications portable, scalable, and cloud-native. This module covers the evolution of deployment, VM vs. container architecture, OCI standards, image registries, Docker and Podman, and networking and storage fundamentals.



From Physical Servers to OpenShift

Each era improved agility, portability, and efficiency — but introduced new operational demands.

1

Physical Servers

Low utilization, slow provisioning

2

Virtual Machines

Heavyweight, resource-intensive

3

Containers

Lightweight, needs orchestration

4

Kubernetes

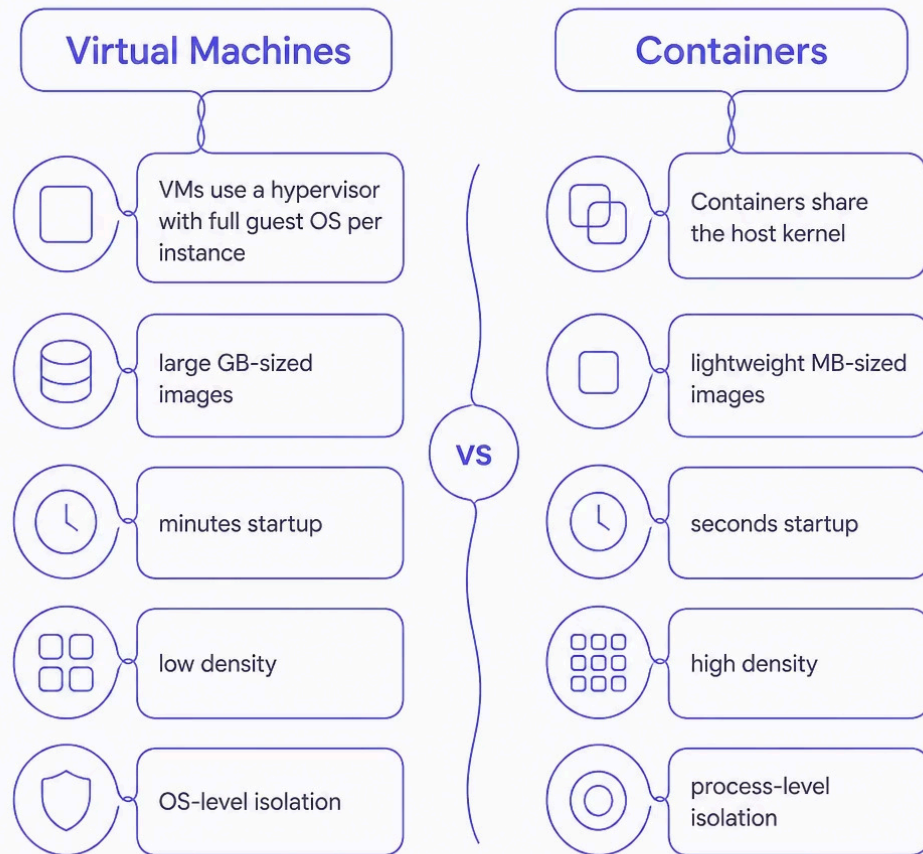
Automated scheduling at scale

5

OpenShift

Enterprise platform automation

Virtual Machines vs. Containers



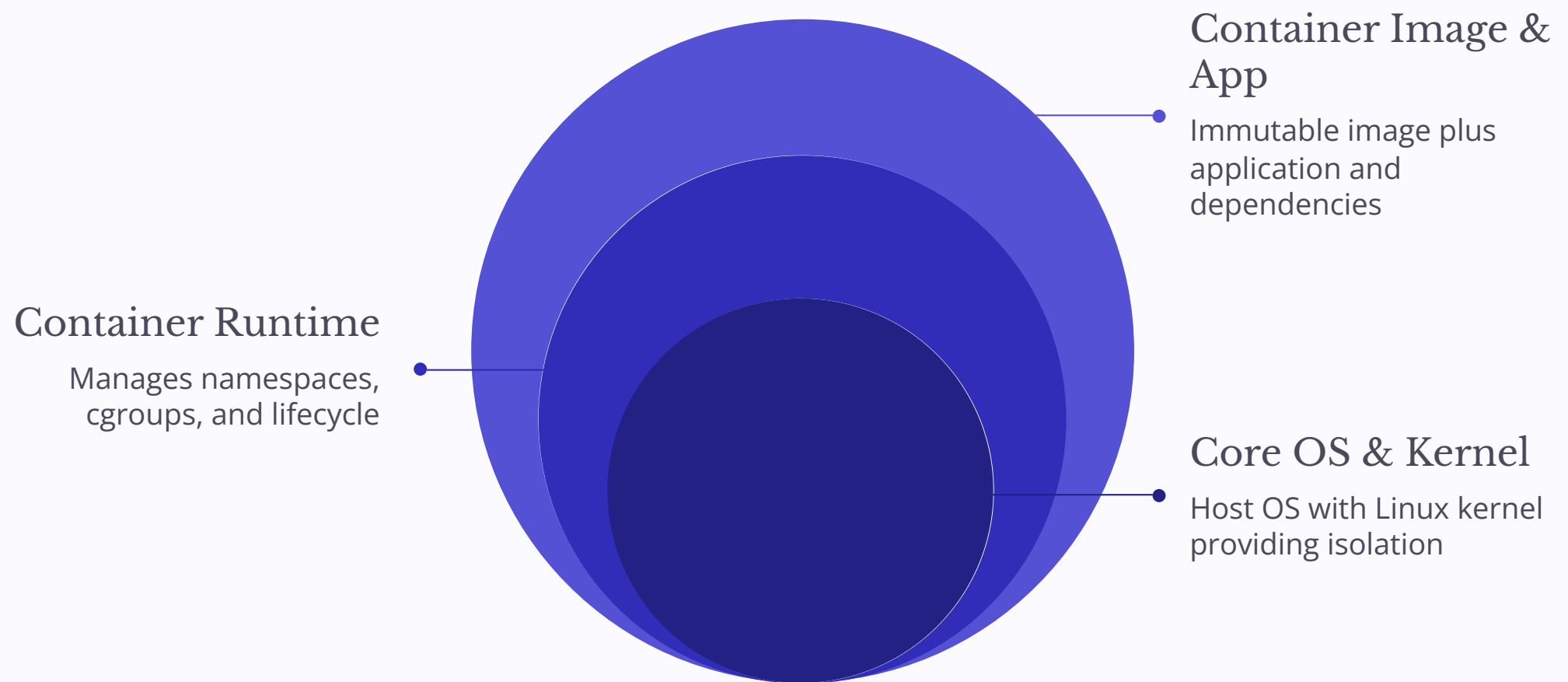
Key Differences

- **VMs** include a full guest OS — heavyweight but strongly isolated
- **Containers** share the host kernel — lightweight, fast, and dense
- Containers start in **seconds** vs. minutes for VMs
- Containers achieve far **higher density** on the same hardware

 OpenShift runs containers — not VMs — maximizing resource efficiency at scale.

Container Architecture Fundamentals

Containers are isolated processes on a shared kernel, built on core Linux technologies.



Each layer builds on the one below — the kernel provides isolation primitives that the runtime uses to launch containers from images.

Namespaces

Isolate process views

cgroups

Limit resource usage

Union FS

Layered image storage

Capabilities

Fine-grained privileges

Seccomp

Filter system calls



STANDARDS

Open Container Initiative (OCI)

OCI defines open standards ensuring containers work consistently across platforms and vendors.



Image Specification

Standard format for container images



Runtime Specification

Defines how containers are executed



Distribution Specification

Standardizes image push and pull

runc

Reference runtime

Docker

Container engine

Podman

Daemonless engine

Quay

Enterprise registry

Container Images and Registries

What Is a Container Image?

An **immutable, layered package** containing application code, runtime libraries, dependencies, and configuration. Layers are cached and shared across images.

❏ **Key distinction:** Images are read-only templates; containers are running instances.

Image Layer Stack



1 Configuration Layer

2 Application Layer

3 Runtime Layer

4 Base OS Layer

Public Registries

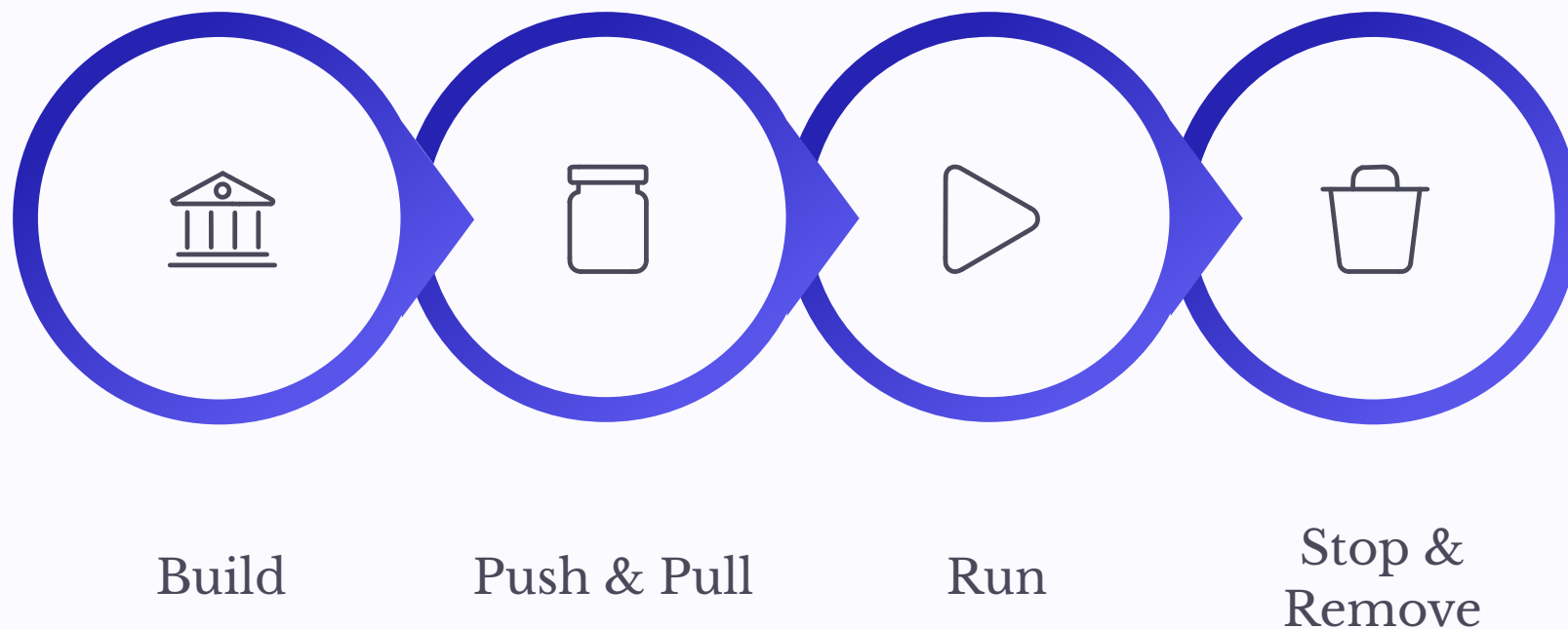
Docker Hub · Quay.io

Private Registries

OpenShift Internal · Enterprise

The Container Lifecycle

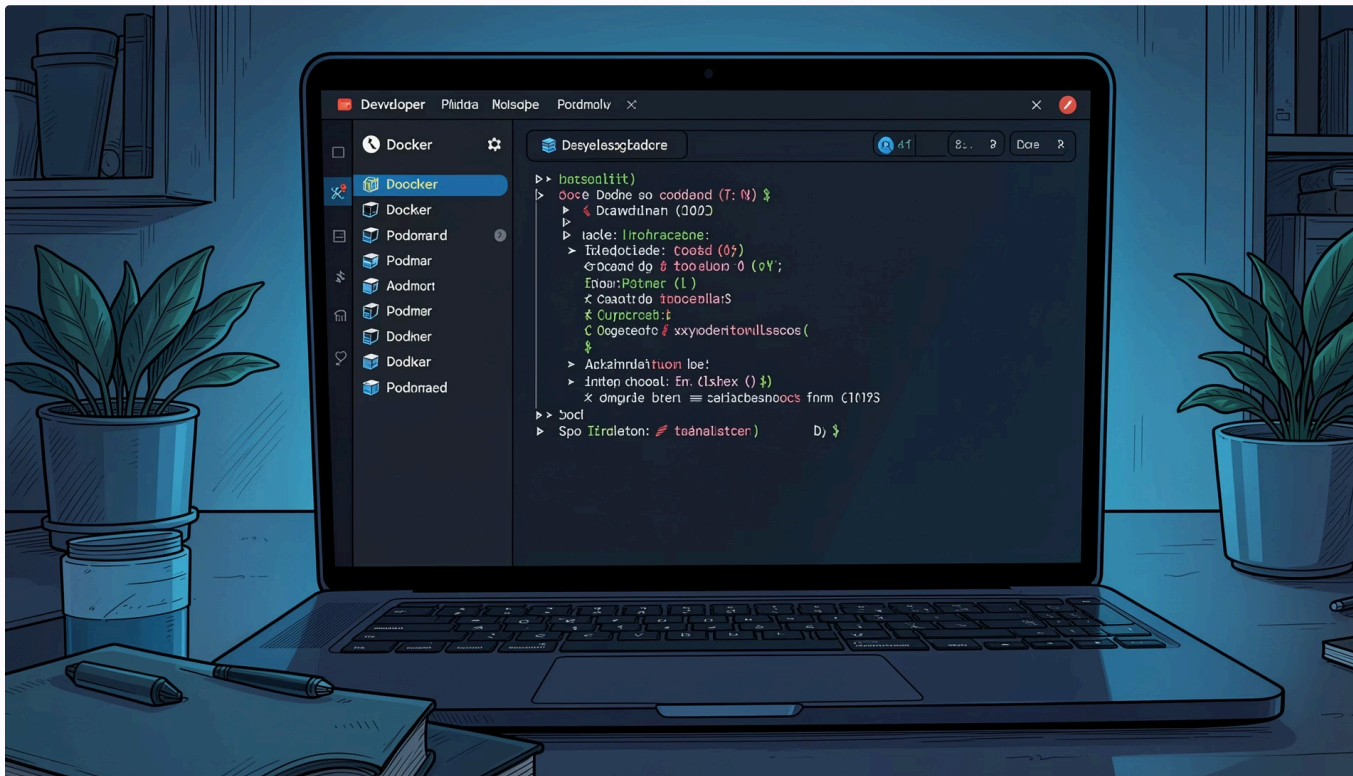
Containers follow a predictable, repeatable lifecycle — from image build to runtime execution and removal. This model enables **immutable infrastructure** and **ephemeral, stateless** workloads.



This pipeline ensures consistent, repeatable deployments — the same image runs identically in dev, test, and production.

❓ Why should containers be treated as disposable rather than permanent servers?

Docker and Podman



Docker

- Client-server architecture with a central daemon
- Mature ecosystem and extensive tooling
- Widely adopted by developers

Podman

- Daemonless — no central service running
- Native rootless execution for security
- OCI-compliant and enterprise-friendly

i OpenShift uses **CRI-O** and aligns closely with Podman's security-first, daemonless model.

Container Networking and Storage

Networking

Each container has an isolated network stack. Containers communicate via virtual networks with DNS-based service discovery.

- Isolated network namespace per container
- Ports published to the host or cluster
- DNS-based service discovery

Storage

Containers are ephemeral — data must outlive the container. Use persistent storage for stateful workloads.

- **Volumes** — managed persistent storage
- **Bind mounts** — host directory mapping
- **PersistentVolume** — OpenShift storage abstraction

📌 **Key principle:** Containers are temporary — design workloads to be stateless and let the platform manage persistence.

HANDS-ON LABS

Lab Roadmap

Five progressive labs build practical skills with Docker and Podman.

01

Registry Setup

Register on Docker Hub or Quay.io

02

Run Containers

Launch containers with `docker run` and `podman run`

03

Manage Containers

List, inspect, stop, and remove containers

04

Build Images

Write a Dockerfile and build with `podman build`

05

Push and Pull

Tag, push to a registry, and pull images

Q & A and Thank you!

Thank you for your attention. We hope this module provided a solid foundation in container fundamentals and their practical application. We're excited to see you apply these concepts.

Now, let's open the floor for your questions and further discussion.

